



BL616L Linux 驱动 移植及使用说明

version : 4.0

Copyright @ 2025

www.bouffalolab.com

版本	描述	时间
1.0	初始版本	2022-06
2.0	修正、格式化	2024-06
3.0	统一 WIFI ONLY 和 WIFI-BLE combo	2024-11
4.0	BL616L	2025-08

目录

1	内核配置.....	4
2	驱动代码修改.....	5
3	驱动交叉编译.....	6
4	固件下载.....	8
5	驱动加载及参数.....	11
6	蓝牙 hci 设备.....	14
7	认证及测试.....	15
8	常见问题.....	17

1 内核配置

CFG80211 模式支持：驱动支持 cfg80211，Linux Kernel CFG80211 相关的 config 都需要打开。

```
CONFIG_FIB_RULES=y
CONFIG_WIRELESS=y
CONFIG_WIRELESS_EXT=y
CONFIG_WEXT_CORE=y
CONFIG_WEXT_PROC=y
CONFIG_WEXT_PRIV=y
CONFIG_CFG80211=m
# CONFIG_NL80211_TESTMODE is not set
# CONFIG_CFG80211_DEVELOPER_WARNINGS is not set
CONFIG_CFG80211_DEFAULT_PS=y
# CONFIG_CFG80211_INTERNAL_REGDB is not set
CONFIG_CFG80211_CRDA_SUPPORT=y
CONFIG_CFG80211_WEXT=y
# CONFIG_CFG80211 is not set
```

支持 iwpriv 命令，需要将 CONFIG_WIRELESS_EXT、CONFIG_WEXT_PRIV、CONFIG_WEXT_CORE 几个宏打开。iwpriv 为驱动支持命令接口，用于产测模式或联网模式下的一些命令。

支持 WEXT 模式，驱动默认不支持 WEXT 模式，如要支持 WEXT，需要在 Makefile 里配置 CONFIG_SUPPORT_WEXT_MODE ?= y，并且 kernel config 中需要打开 CONFIG_CFG80211_WEXT，采用 kernel 提供的 WEXT 兼容方式。

2 驱动代码修改

部分主控平台的 SDIO 接口，为了检测到 SDIO 接口的 BL616L，需要在本驱动入口合入 SDIO detect 或 power on/off 相关函数，以 T31 为例，分别在 bl_mod_init() 和 bl_mod_exit() 加入相关函数。

```
#define WLAN_SDIO_INDEX 1
int jzmmc_manual_detect(int index, int on);

int platform_wifi_power_on(void)
{
    jzmmc_manual_detect(WLAN_SDIO_INDEX, 1);
    return 0;
}

int platform_wifi_power_off(void)
{
    jzmmc_manual_detect(WLAN_SDIO_INDEX, 0);
    return 0;
}

static int __init bl_mod_init(void)
{
    BL_DBG(BL_FN_ENTRY_STR);
    bl_print_version();
    platform_wifi_power_on();
    return bl_platform_register_drv();
}

static void __exit bl_mod_exit(void)
{
    BL_DBG(BL_FN_ENTRY_STR);
    bl_platform_unregister_drv();
    platform_wifi_power_off();
}
```

3 驱动交叉编译

编译前先检查 toolchain 路径是否已加入环境变量中。

BL616L 芯片有 USB 和 SDIO 接口，以透传模式跟上位主控 CPU 通信，再根据是否要 WIFI 和 BLE 共存，修改驱动 Makefile 开头部分，选定编译的驱动类型，只能设定一个类型为 y。选定的驱动类型和后续使用固件类型要相匹配。可参考后续章节-固件类型。

```
#####  
##### set only one following option to y, set others to n.  
#####  
##WIFI-SDIO only  
CONFIG_BL_SDIO ?= n  
##WIFI-SDIO-BLE-SDIO combo  
CONFIG_BL_BTSDU ?= y  
##WIFI-SDIO-BLE-UART combo  
CONFIG_BL_BTUART ?= n  
##WIFI-USB only  
CONFIG_BL_USB ?= n  
##WIFI-USB-BLE-USB combo  
CONFIG_BL_BTUSB ?= n
```

驱动 Makefile 里直接指定 ARCH、CROSS_COMPILE、KERNELDIR 路径。

```
##cross compile options  
ARCH ?= arm  
CROSS_COMPILE ?= arm-gcc6.3-linux-uclibceabi-
```

```
KERNELDIR ?= /lib/modules/`uname -r`/build
```

以上准备好后，就可以执行编译命令：`make clean & make`

如果不想修改 Makefile 里的驱动类型、ARCH、CROSS_COMPILE、KERNELDIR，也可以直接在 make 命令行指定，比如：

```
export          PATH=/software/pc/toolchain/mips-gcc540-uclibc0.9.33.2-64bit-
r3.3.0.smaller/bin/

ARCH=mips

CROSS_COMPILE=mips-linux-uclibc-

KERNELDIR=/Software/os/kernel/

make          ARCH=${ARCH}          CROSS_COMPILE=${CROSS_COMPILE}
KERNELDIR="$KERNELDIR" clean

make          CONFIG_BL_SDIO=y          ARCH=${ARCH}
CROSS_COMPILE=${CROSS_COMPILE} KERNELDIR="$KERNELDIR"
```

其他注意事项：

1) 如果 BLE 采用 UART 接口，则使用 kernel 自带的 HCI UART 驱动。

2) WPA3 支持：

如果 kernel $\geq$ 4.17, 驱动不需要做任何配置，可以直接支持 WPA3。

如果 kernel $<$ 4.17, 则 kernel 需要打 patch，另外，驱动需要在 Makefile 里把 `CONFIG_WPA3_SUPPORT = y`

3) 编译时可能会遇到一些文件权限或 windows 格式问题，可以用 `dos2unix & chmod` 解决，如下：

```
dos2unix  *  */*
```

```
chmod 777 *.sh
```

4 固件下载

4.1 固件类型

BL616L 芯片有 USB 和 SDIO 接口，以透传模式跟上位主控 CPU 通信。BL616L 发布的安装包有两种工作模式的固件，分别用于驱动的联网模式和产测模式。

实际产品使用时，根据接口类型、WiFi 和 BLE 是否共存来选择联网固件和产测固件各一个。

比如 USB 接口、WiFi only，则使用 bl616l_usb_wlan.bin 和 bl616l_usb_mp.bin 来作为联网固件和产测固件。

比如 SDIO 接口、WiFi 和 BLE combo，WiFi 和 BLE 都使用 SDIO 来通信，则使用 bl616l_sd_combo_ble_sdu.bin 和 bl616l_sd_mp.bin 来作为联网固件和产测固件。

类型	驱动 Makefile 选择相应类型?=y	联网固件	产测固件
WIFI-USB only	CONFIG_BL_USB WIFI over USB	bl616l_usb_wlan.bin	bl616l_usb_mp.bin
WIFI-USB-BLE-USB combo	CONFIG_BL_BTUSB WIFI over USB BLE over USB	bl616l_usb_combo.bin	
WIFI-SDIO only	CONFIG_BL_SDIO WIFI over SDIO	bl616l_sd_wlan.bin	bl616l_sd_mp.bin
WIFI-SDIO-BLE- SDIO combo	CONFIG_BL_BTSDU WIFI over SDIO	bl616l_sd_combo_ble_sdu.bin	

	BLE over SDIO		
WIFI-SDIO-BLE- UART combo	CONFIG_BL_BTUART WIFI over SDIO BLE over UART	bl616l_sd_combo_ble_uart.bin	

产测固件是 WIFI 和 BLE combo 的产测固件，支持 WIFI 和 BLE 产测命令。产测命令，参考产测文档。

4.2 固件下载方式

4.2.1 通过/lib/firmware 下载固件

编译驱动时，确保 Makefile 里 `CONFIG_BL_DNLD_FWBIN=y`。将两类固件都放到宿主机 Linux 的 `/lib/firmware/` 目录下，驱动安装时会根据工作模式（联网或产测）及接口类型（USB 或 SDIO）从 `/lib/firmware` 目录选择对应固件下载到 BL616 芯片。

4.2.2 转为数组后直接编译进驱动

编译驱动时,确保 Makefile 里 `CONFIG_BL_DNLD_FWBIN=n`，将固件拷贝到驱动源码根目录，如下命令转换固件为数组来更新驱动里对应的 c 文件，然后编译驱动。

五选一

```
xxd -g 1 -i bl616l_usb_wlan.bin > bl_fwbin.c
```

```
xxd -g 1 -i bl616l_usb_combo.bin > bl_fwbin.c
```

```
xxd -g 1 -i bl616l_sd_wlan.bin > bl_fwbin.c
```

```
xxd -g 1 -i bl616l_sd_combo_ble_sdu.bin > bl_fwbin.c
```

```
xxd -g 1 -i bl616l_sd_combo_ble_uart.bin > bl_fwbin.c
```

二选一

```
xxd -g 1 -i bl616l_usb_mp.bin > bl_mpfwbin.c
```

```
xxd -g 1 -i bl616l_sd_mp.bin > bl_mpfwbin.c
```

Bouffalo Lab

5 驱动加载及参数

BL616L 的 SDIO 接到主控后，上电识别，dmesg 能看到 "new high speed SDIO card at address" 的消息。

BL616L 的 USB 接到主控后，上电识别，lsusb 能看到 ID 349b:6160 的设备，dmesg 也能看到相应的识别消息。

驱动编译好后，在相应文件夹里(usb or sdio)会生成 bl_usb_drv.ko or bl_sdio_drv.ko.

联网模式安装驱动命令：

```
insmod bl_usb_drv.ko
```

```
insmod bl_sdio_drv.ko
```

产测模式安装驱动命令：

```
insmod bl_usb_drv.ko mp_mode=1
```

```
insmod bl_sdio_drv.ko mp_mode=1
```

5.1 参数说明

mp_mode:

1, 表示产测模式，驱动安装后会工作在产测模式，射频测试一般采用这种模式，具体参考《BL616L Linux 产测命令用户手册》。产测模式下，需要确保 kernel 配置好 `CONFIG_WIRELESS_EXT=y` 和 `CONFIG_WEXT_PRIV=y`，支持 iwpriv 命令。

cal_data_cfg:

校准相关参数，如果选择将 capcode, power_offset 等校准值不烧写进 Efuse，而是写进 rf_para.conf，则在联网模式加载驱动时加载此参数，使用方法如下：

```
insmod bl_usb_drv.ko cal_data_cfg=$abspath/rf_para.conf
```

\$abspath 表示绝对路径。

tx_pwr_cfg:

TX 功率配置相关参数，可以在 `tx_pwr.conf` 配置各模式速率的发送功率。在驱动里也有一个发送功率表，客户也可以选择驱动里修改。

```
insmod bl_usb_drv.ko tx_pwr_cfg=$path/tx_pwr.conf
```

`tx_pwr.conf` 存放路径规则同 `rf_para.conf` 文件存放规则。

wifi_mac:

用参数设置 mac address，不设置则使用芯片 Efuse 里的 mac 地址。

```
insmod bl_usb_drv.ko wifi_mac=xx:xx:xx:xx:xx:xx
```

Efuse 中有 2-3 个 slot 来存 mac 地址，第一个 slot 为博流芯片出厂时写的 mac 地址，后续空余 slot 可以给终端用户通过产测驱动和命令来写入自己的 mac 地址（如果客户需要写入自己的地址的情况下）。固件运行时，会选择最后写入 mac 地址来使用及上报给驱动和内核。

5.2 功率表参数说明

驱动自带默认国家功率表，在 `bl_cfgfile.c` 的 `bl_country_default`。请根据实际射频测试结果填写。

如果不想一次性固化功率表在驱动中或者针对不同板子使用差异性功率表，可以生成如驱动源码 `conf` 目录下的 `country_tx_pwr.cfg`（多个国家功率表）或 `tx_pwr.cfg`（一张通用功率表适用多个国家）。后续驱动加载时通过参数来加载功率表配置文件。

1) `country_pwr_cfg` 指定 `country_tx_pwr.cfg` 的绝对路径。

```
insmod bl_usb_drv.ko country_pwr_cfg=/home/drv/conf/country_tx_pwr.conf
```

指定各个国家的功率表，及当前国家码。

2) `tx_pwr_cfg` 指定 `tx_pwr.cfg` 的绝对路径。

```
insmod usb/bl_usb_drv.ko tx_pwr_cfg=/home/drv/conf/tx_pwr.conf
```

直接指定一个通用功率表，应用于所有国家码。

3) `cal_data_cfg` 可以指定一个通用功率表（类似 `tx_pwr_cfg`，应用于所有国家码）和信道功率偏移校准表等多种配置，使用绝对路径，格式如 `rf_para-full.conf` 文件。可以根据需求来修改配置项的对应值，或删除里面不需要的配置项。

```
insmod usb/bl_usb_drv.ko cal_data_cfg=/home/drv/conf/rf_para.conf
```

以上三者，一般根据需要使用一种或完全不使用，在同时使用多个的情况下，后者包含的功率表覆盖前者包含的功率表。

`cal_data_cfg` 指定的配置文件中，额外包含信道功率校准表等其他校准值，可以在产线生产时或模组生产时写进 `efuse` 的，那么就不用驱动加载时用参数指定配置文件来加载。

配置 `country_pwr_cfg` 功率表或使用驱动自带的国家功率表后，后续国家码设置生效时，驱动会选择对应的功率配置下发到固件生效。如果不设置国家码，则默认使用 CN 对应的功率表。如果使用了 `tx_pwr_cfg` 或 `cal_data_cfg` 设置功率表，则所有国家使用一样的功率表。

6 蓝牙 hci 设备

针对 WIFI-SDIO-BLE-UART 类型固件和驱动，**必须使用 4 线 UART**，需要执行 `hciattach` 来产生 hci 设备(`hciattach -s 115200 /dev/ttyUSB0 any 115200 flow nosleep`)。具体参考《BLUEZ 交叉编译及使用.pdf》。

针对 WIFI-USB-BLE-USB combo 或 WIFI-SDIO-BLE-SDIO 类型固件和驱动，加载相应驱动后，就可以通过 `hciconfig` 看到 hci 设备。

现在 WiFi-BLE combo 固件，内部只带了低功耗蓝牙 BLE 的相关功能，在 Linux 端上支持低功耗蓝牙 BLE 相关命令及功能，不支持 Linux 上的任何经典蓝牙 BT 相关命令及功能。

7 认证及测试

7.1 WIFI 认证

比如 SRRC、CE、FCC 等 WIFI 认证，按需求使用联网模式或产测模式来认证。如需使用产测模式，请参考《BL616L_Linux 产测命令用户手册》。

针对 WIFI-BLE combo 的固件，WIFI 自适应认证时请确保 BLE 是关闭的。

7.2 蓝牙 BQB 认证

7.2.1 WIFI-SDIO-BLE-UART combo

针对使用 WIFI-SDIO-BLE-UART 固件的 BL616L，主控是通过 UART 来跟 BL616L 进行蓝牙通信的。过认证前，把 BL616L 的 UART 跟主控之间硬件断开，将 BL616L UART 飞线出来，使用 USB 转串口的 dongle（例如 FT232 芯片）转接，USB dongle 插到测试仪器上。BL616L 的天线部分用传导线连接到测试仪器上。

主控系统启动后按联网模式加载驱动，会自动下载 BL616L 联网的 combo 固件，但主控系统后续不执行 hciattach 来把 BL616L UART 接入主控，不执行其他任何的 WiFi 动作，比如连线等。如果加载驱动后系统自动创建了 wlan0 或 ap0 的网络接口，可以执行 ifconfig wlan0 down 或 ifconfig ap0 down 来禁止。

认证人员及测试仪器会按测试标准及各个 case 来发送 HCI 指令，经 UART 发给 BL616L 完成测试。

7.2.2 WIFI-SDIO-BLE-SDIO combo 和 WIFI-USB-BLE-USB combo

可以考虑使用 WIFI-SDIO-BLE-UART combo 来替代过认证，按上述 WIFI-SDIO-BLE-UART combo 的方式操作。

如果必须使用 WIFI-SDIO-BLE-SDIO combo 和 WIFI-USB-BLE-USB combo 自身来过认证，则需要满足一些条件和操作。驱动源码目录下的 bl_hci.c 编译生成一个 Linux 应用程序 bl_hci，运行在主控命令行终端且占用终端的输入输出，任何输入到此终端的内容必须是 16 进制格式的 HCI 命令---由测试仪器发出，被 bl_hci 应用程序接收，转给驱动再转到固件处理。bl_hci 会把来自固件的所有 HCI 响应按 16 进制格式打印到此终端---由测试仪器接收。

主控命令行终端是运行在串口等可以被测试仪器识别的接口上。主控系统 `bl_hci` 运行时，不能往终端上输入输出任何信息，避免扰乱 HCI 命令收发。

Bouffalo Lab

8 常见问题

8.1 版本查看

如果主控 Linux 系统支持 `iwpriv` 命令，则使用命令 `iwpriv wlan0 version Signed` 来查看。

如果主控不支持 `iwpriv` 命令，则在驱动加载的日志中自行查找如下开头的字段：

BL FW version: 6.16.0.5-105, Signed: 6ffdb331

BL Wlan Driver version: 102-012, Signed: 8f1fff5

8.2 报问题要求

报问题时，请描述主控硬件类型，Linux 版本，BL616L 驱动和固件版本，系统启动日志及加载驱动的日志，问题相关的日志，尽量详细描述问题发生的无线环境场景和问题复现操作步骤。

如果是驱动导致内核 `crash` 的异常问题，请同时将驱动反汇编的输出发出，使用交叉编译器的 `objdump`，类似如下命令产生反汇编输出：

```
xxx-objdump -s -S -x -D -g bl_sdio_drv.ko > bl_sdio_drv.ko.dump.txt
```

8.3 国家码

配置 Kernel 支持 `country code`，打开下面 2 个宏

```
CONFIG_CFG80211_REQUIRE_SIGNED_REGDB
```

```
CONFIG_CFG80211_USE_KERNEL_REGDB_KEYS
```

`sforshee.hex` 放到 `linux-4.19.123/net/wireless/certs/` 目录下(4.15 后的内核已自带)，

`regulatory.db.p7s` 和 `regulatory.db` 两个文件放到 `/lib/firmware`，确保这文件在内核启动过程中是能访问到的。如果内核启动过程中不能访问到 `/lib/firmware` 下的这两个文件，请把 `cfg80211` 变成 `ko`，在 `insmod WIFI 驱动前` `modprobe cfg80211.ko`。

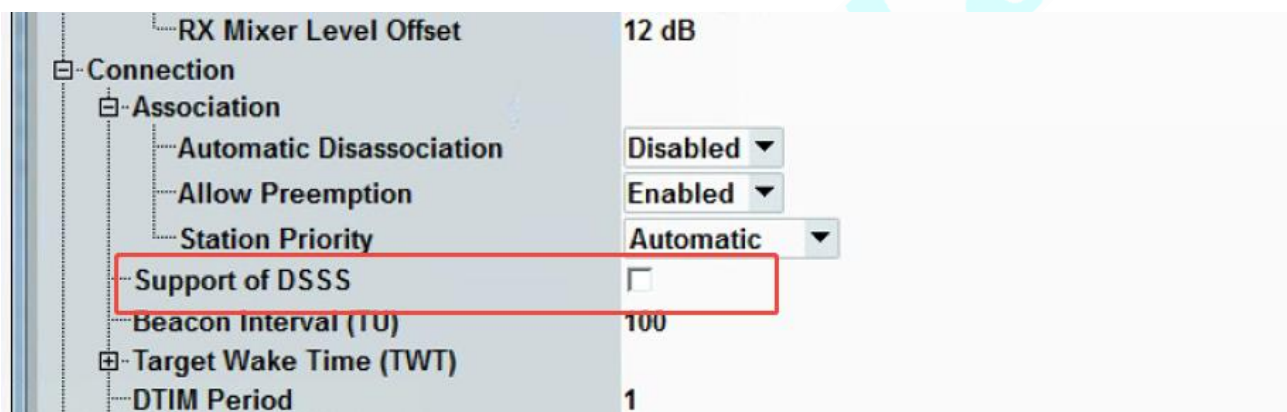
如果驱动加载时参数 `country_pwr_cfg` 指定的配置文件里指定了 `COUNTRY` 项，则会设置驱动的 `country_code`。

如果驱动加载时用参数 `country_code`，也可以指定驱动的 `country_code`，会覆盖参数 `country_pwr_cfg` 的配置文件指定的 `country_code`。

国家码一般是应用层（比如 `wpa_supplicant` 或 `hostapd` 或其他方式）配置到内核，内核通知驱动，并覆盖驱动原有的 `country_code`（不管是历史上内核设置给驱动的还是驱动加载时参数设置的 `country_code`）。

不同的国家码，会影响到功率表，参考《功率表参数说明》章节。

8.4 CMW 仪器起 11AX AP 要支持 DSSS 才能被连接



8.5 BL616L 支持的接口数和设备数

BL616L 支持起 2 个网络接口，一般只会单独做 STA 来使用，或单独做 AP 来使用，或串行顺序的做 STA 和 AP 来使用。

如果使用过程中是 AP+STA 共存使用，强制 AP 信道跟随 STA。

BL616L 做 AP 时，下面最多可以带 4 个 STA，分享带宽。

8.6 SDIO 时钟及工作问题

BL616L SDIO 接口与主控硬件接通后，主控 Linux 系统开启支持对应 mmc 接口，BL616L 驱动移植到板子的系统里。板子上电后，注意看主控系统给 SDIO 接口的配置的时钟，主控是否识别对应的 mmc 接口，类似下图的打印，则识别 BL616L

SDIO 是顺利，即认卡成功。

```

439 [ 3.269740] Segment Routing with IPv6
440 [ 3.269810] NET: Registered protocol family 17
441 [ 3.269826] NET: Registered protocol family 15
442 [ 3.269847] can: controller area network core
443 [ 3.269884] NET: Registered protocol family 29
444 [ 3.269890] can: raw protocol
445 [ 3.269897] can: broadcast manager protocol
446 [ 3.269907] can: netlink gateway - max_hops=1
447 [ 3.270048] Bluetooth: RFCOMM socket layer initialized
448 [ 3.270074] Bluetooth: RFCOMM ver 1.11
449 [ 3.270085] Bluetooth: HIDP (Human Interface Emulation) ver 1.2
450 [ 3.270094] Bluetooth: HIDP socket layer initialized
451 [ 3.270129] [BT_RFKILL]: Enter rfkill_rk_init
452 [ 3.270135] [WLAN_RFKILL]: Enter rfkill_wlan_init
453 [ 3.270584] Key type dns_resolver registered
454 [ 3.271611] Loading compiled-in X.509 certificates
455 [ 3.272323] pstore: Using crash dump compression: deflate
456 [ 3.272913] rga2 ff440000.rga: Adding to iommu group 3
457 [ 3.273095] rga: rga2, irq = 32, match scheduler
458 [ 3.273465] rga: rga2 hardware loaded successfully, hw_version:3.6.92812.
459 [ 3.273521] rga: rga2 probe successfully
460 [ 3.273657] rga_iommu: IOMMU binding successfully, default mapping core[0x4]
461 [ 3.273805] rga: Module initialized. v1.2.26
462 [ 3.278305] mmc1: SDHCI controller on ff870000.mmc [ff870000.mmc] using ADMA
463 [ 3.289304] mmc_host mmc0: Bus speed (slot 0) = 50000000Hz (slot req 50000000Hz, actual 50000000Hz div = 0)
464 [ 3.289600] mmc0: new high speed SDIO card at address 0001
465 [ 3.291698] rockchip-dmc dmc: leakage=19
466 [ 3.291740] rockchip-dmc dmc: leakage-volt-sel=1
467 [ 3.291954] rockchip-dmc dmc: avs=0
468 [ 3.291973] rockchip-dmc dmc: current ATF version 0x101
469 [ 3.292434] rockchip-dmc dmc: normal_rate = 780000000
470 [ 3.292443] rockchip-dmc dmc: reboot_rate = 1332000000
471 [ 3.292450] rockchip-dmc dmc: suspend_rate = 324000000
472 [ 3.292458] rockchip-dmc dmc: video_4k_rate = 780000000
473 [ 3.292466] rockchip-dmc dmc: video_4k_10b_rate = 780000000

```

系统启动后续加载驱动，注意是否一直打印"FW Mark"，而不打印"FW Mark B:42"，此时表示固件未正常从主控通过 SDIO 接口下载到 BL616L 的 RAM 运行，驱动不会继续往下执行去跟 BL616L 通信及后续任何功能。此时需要看时钟是不是太高，50M 时钟是上限，在板子有潜在可能因素干扰到 SDIO 的情况下，可以将时钟降低到 25M，再来跑通整个驱动加载及 WIFI 工作整个流程，跑通且稳定后，再逐步提高时钟和检查并消除板子上潜在干扰源。

驱动加载时，打印"FW Mark B:42"，则表示固件下载就完成。如果后续基于 SDIO 通信也可能由于干扰而不稳定，进而出异常（如下图中的异常 invalid skb->len，但不局限于下图），那么也要考虑降低时钟来尝试或消除干扰源再尝试。

```
622 [ 25.031493] bl_sdio_drv: loading out-of-tree module taints kernel.
623 [ 25.042468] BL Wlan Driver version 102.012, Signed: 0x44308ec
624 [ 25.042850] bl_sdio_probe info: vendor=0x424C device=0x0606 class=0 function=1
625 [ 25.042855] max_blksiz=0->512, enable_timeout=0->200
626 [ 25.042860] after1 max_blksiz=512, enable_timeout=200
627 [ 25.043031] bl_device_init before write sdio_ireg=0x0, ret=0
628 [ 25.043067] bl_device_init after write sdio_ireg=0x10, ret=0
629 [ 25.043103] Enter bl_sdio_run_fw...
630 [ 25.143159] >>> bl_sdio_checkversion()
631 [ 25.143308] CMD check version len 0
632 [ 25.143670] <<< bl_sdio_checkversion()
633 [ 25.143674] Enter bl_sdio_download_firmware...
634 [ 25.143677] fw_name:bl616_sd_wlan.bin
635 [ 25.147061] Downloading image (269296 bytes)
636 [ 25.147082] >>> firmware_download_head()
637 [ 25.147779] [RSP] bootheader_load response OK
638 [ 25.147785] <<< firmware_download_head()
639 [ 25.147789] no sign field detected
640 [ 25.147792] no sign field detected
641 [ 25.147797] >>> firmware_download_encryptiondata()
642 [ 25.147800] no encrypt field detected
643 [ 25.147804] >>> firmware_download_segment_piece()
644 [ 25.148400] [RSP] section_header response OK
645 [ 25.199348] <<< firmware_download_segment_piece()
646 [ 25.199668] [RSP] checkimage response OK
647 [ 25.199974] [RSP] runimage response OK
648 [ 25.199988] Download fw time: 52906
649 [ 25.200100] <<< bl_sdio_init()
650 [ 25.200517] bl_sdio_drv mmc0:0001:1: Direct firmware load for bl_wifi_mac.ini failed with error -2
651 [ 25.200525] bl_parse_configfile: no bl_wifi_mac.ini, (-2)
652 [ 25.200541] bfl random mac:
653 [ 25.200541] 7c:b9:4c:de:c0:8d
654 [ 25.201120] Wait firmware...
655 [ 25.201178] FW Mark
656 [ 25.301307] FW Mark
657 [ 25.401534] FW Mark B:42
658 [ 25.502063] bl_decode_rx_packet: invalid skb->len: 0
659 [ 25.502087] bl_rx_process error
660 [ 25.502109] info: CFG reg val = 0
661 [ 25.502129] info: write success
```

一般先尝试 20-30M 的时钟配置，稳定后，再提高时钟来尝试，得到一个稳定时钟配置。

8.7 WiFi 抓包

```
sudo apt install aircrack-ng
```

```
sudo systemctl stop NetworkManager
```

```
sudo airmon-ng start wlan0
```

```
sudo wireshark
```

wireshark 启动后，看到无线网络接口名字会变成以 mon 结尾的，双击选它来抓

包。选 Channel 的地方，选下 WiFi 无线信道。

8.8 mac 地址

- 1) 根据驱动加载参数 `wifi_mac` 指定 mac 地址，优先级最高，覆盖其它 mac 地址，但一般不使用此种方式。
- 2) 固件自动读取 Efuse 里的 mac 地址。Efuse 里默认自带一个博流的 mac 地址。客户产线可以使用产测命令写入自己的 mac 地址。

8.9 Linux 内核集成编译驱动

将驱动源码放到 `drivers/net/wireless` 目录下，在 `drivers/net/wireless/Kconfig` 里添加 `source "drivers/net/wireless/BL616L_HostDriver/Kconfig"`

```
# SPDX-License-Identifier: GPL-2.0-only
#
# wireless LAN device configuration
#

menuconfig WLAN
    bool "wireless LAN"
    depends on !S390
    depends on NET
    select WIRELESS
    default y
    help
        This section contains all the pre 802.11 and 802.11 wireless
        device drivers. For a complete list of drivers and documentation
        on them refer to the wireless wiki:

        https://wireless.wiki.kernel.org/en/users/Drivers

if WLAN

source "drivers/net/wireless/admtek/kconfig"
source "drivers/net/wireless/ath/kconfig"
source "drivers/net/wireless/atmel/kconfig"
source "drivers/net/wireless/broadcom/kconfig"
source "drivers/net/wireless/cisco/kconfig"
source "drivers/net/wireless/intel/kconfig"
source "drivers/net/wireless/intersil/kconfig"
source "drivers/net/wireless/marvell/kconfig"
source "drivers/net/wireless/mediatek/kconfig"
source "drivers/net/wireless/microchip/kconfig"
source "drivers/net/wireless/ralink/kconfig"
source "drivers/net/wireless/realtek/kconfig"
source "drivers/net/wireless/rsi/kconfig"
source "drivers/net/wireless/st/kconfig"
source "drivers/net/wireless/ti/kconfig"
source "drivers/net/wireless/zydas/kconfig"
source "drivers/net/wireless/quantenna/kconfig"
source "drivers/net/wireless/BL616L_HostDriver/kconfig"

config PCMCIA_RAYCS
    tristate "Aviator/Raytheon 2.4GHz wireless support"
    depends on PCMCIA
    select WIRELESS_EXT
    select WEXT_SPY
    select WEXT_PRIV
    help
```

在 drivers/net/wireless/Makefile 里添加

obj-\$(CONFIG_BL616L) += BL616L_HostDriver/


```
# SPDX-License-Identifier: GPL-2.0
#
# Makefile for the Linux wireless network device drivers.
#
obj-$(CONFIG_WLAN_VENDOR_ADMTEK) += admtek/
obj-$(CONFIG_WLAN_VENDOR_ATH) += ath/
obj-$(CONFIG_WLAN_VENDOR_ATMEL) += atmel/
obj-$(CONFIG_WLAN_VENDOR_BROADCOM) += broadcom/
obj-$(CONFIG_WLAN_VENDOR_CISCO) += cisco/
obj-$(CONFIG_WLAN_VENDOR_INTEL) += intel/
obj-$(CONFIG_WLAN_VENDOR_INTERSIL) += intersil/
obj-$(CONFIG_WLAN_VENDOR_MARVELL) += marvell/
obj-$(CONFIG_WLAN_VENDOR_MEDIATEK) += mediatek/
obj-$(CONFIG_WLAN_VENDOR_MICROCHIP) += microchip/
obj-$(CONFIG_WLAN_VENDOR_RALINK) += ralink/
obj-$(CONFIG_WLAN_VENDOR_REALTEK) += realtek/
obj-$(CONFIG_WLAN_VENDOR_RSI) += rsi/
obj-$(CONFIG_WLAN_VENDOR_ST) += st/
obj-$(CONFIG_WLAN_VENDOR_TI) += ti/
obj-$(CONFIG_WLAN_VENDOR_ZYDAS) += zydas/
obj-$(CONFIG_WLAN_VENDOR_QUANTENNA) += quantenna/
obj-$(CONFIG_BL616L) += BL616L_HostDriver/

# 16-bit wireless PCMCIA client drivers
obj-$(CONFIG_PCMCIA_RAYCS) += ray_cs.o
obj-$(CONFIG_PCMCIA_WL3501) += wl3501_cs.o

obj-$(CONFIG_USB_NET_RNDIS_WLAN) += rndis_wlan.o
obj-$(CONFIG_MAC80211_HWSIM) += mac80211_hwsim.o

obj-$(CONFIG_VIRT_WIFI) += virt_wifi.o
root@raspberrypi:~/rpi-kernel-usbmon/drivers/net/wireless#
```

在对应平台的 defconfig 添加 CONFIG_BL616L=m,

CONFIG_BL_USB

CONFIG_BL_BTUSB

CONFIG_BL_BTSDU

CONFIG_BL_BTUART

CONFIG_BL_SDIO

以上四项驱动类型选一个添加到 defconfig 里。

还可以通过 make menuconfig 来指定以上编译选项，Device Drivers->Network device support->Wireless LAN->BL616L wireless driver 选为 M 或*，再从 BL616L WIFI BLE configuration 里选一种驱动类型。

```
.config - Linux/arm 5.11.22 Kernel Configuration
```

```
> Device Drivers > Network device support > Wireless LAN
```

wireless LAN

Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [] excluded <M> module <> module capable

```

A(-)
[*] Mediatek devices
<M> Mediatek MT7601U (USB) support
<M> Mediatek MT76x0U (USB) support
<> Mediatek MT76x0E (PCIe) support
<> Mediatek MT76x2E (PCIe) support
<M> Mediatek MT76x2U (USB) support
<> Mediatek MT7603E (PCIe) and MT76x8 WLAN support
<> Mediatek MT7615E and MT7663E (PCIe) support
<> Mediatek MT7663U (USB) support
<> Mediatek MT7663S (SDIO) support
<> Mediatek MT7915E (PCIe) support
[*] Microchip devices
<> Atmel WILC1000 SDIO (WiFi only)
<> Atmel WILC1000 SPI (WiFi only)
[*] Ralink devices
<M> Ralink driver support --->
[*] Realtek devices
<> Realtek 8180/8185/8187SE PCI support
<M> Realtek 8187 and 8187B USB support
<M> Realtek rtlwifi family of devices --->
<M> RTL8723AU/RTL8188[CR]U/RTL819[12]CU (mac80211) support
[ ] Include support for untested Realtek 8xxx USB devices (EXPERIMENTAL)
<> Realtek 802.11ac wireless chips support ----
[*] Redpine Signals Inc devices
<> Redpine Signals Inc 91x WLAN driver support
[*] STMicroelectronics devices
<> CWL200 WLAN support
[*] Texas Instrument devices
<> TI wl1251 driver support
<> TI wl12xx support
<> TI wl18xx support
<> TI wlcore support
[*] ZyDAS devices
<M> USB ZD1201 based wireless device support
<M> ZyDAS ZD1211/ZD1211B USB-wireless support
[ ] ZyDAS ZD1211 debugging
[*] Quantenna wireless cards support
<> Quantenna qsr1000/qsr2000/qsr10g PCIe support
<M> BL616L wireless driver
    BL616L WIFI BLE configuration (WIFI over USB and BLE over USB combo.) --->
<M> Simulated Radio testing tool for mac80211
<M> Wireless RNDIS USB support
<> Wifi wrapper for ethernet drivers

```

<Select> < Exit > < Help > < Save > < Load >

.config - Linux/arm 5.11.22 Kernel Configuration

> Device Drivers > Network device support > Wireless LAN

wireless LAN

Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu ----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [] excluded <M> module <> module capable

```

A(-)
[*] MediaTek devices
<M>  MediaTek MT7601U (USB) support
<M>  MediaTek MT76x0U (USB) support
<>   MediaTek MT76x0E (PCIe) support
<>   MediaTek MT76x2E (PCIe) support
<M>  MediaTek MT76x2U (USB) support
<>   MediaTek MT7603E (PCIe) and MT76x8 WLAN support
<>   MediaTek MT7615E and MT7663E (PCIe) support
<>   MediaTek MT7663U (USB) support
<>   MediaTek MT7663S (SDIO) support
<>   MediaTek MT7915E (PCIe) support
[*] Microchip devices
<>   Atmel WILC1000 SDIO (WiFi only)
<>   Atmel WILC1000 SPI (WiFi only)
[*] Ralink devices
<M>  Ralink driver support --->
[*] Realtek devices
<>   Realtek 8180/8185/8187SE PCI support
<M>   Realtek 8187 and 8187B USB support
<M>   Realtek rtlwifi family of devices --->
<M>   RTL8723AU/RTL8188[CR]U/RTL819[12]CU (mac80211) support
[ ]   Include support for untested Realtek 8xxx USB devices (EXPERIMENTAL)
<>   Realtek 802.11ac wireless chips support ----
[*] Redpine Signals Inc devices
<>   Redpine Signals Inc 91x WLAN driver support
[*] STMicroelectronics devices
<>   CW1200 WLAN support
[*] Texas Instrument devices
<>   TI wl1251 driver support
<>   TI wl12xx support
<>   TI wl18xx support
<>   TI wlcore support
[*] ZyDAS devices
<M>   USB ZD1201 based wireless device support
<M>   ZyDAS ZD1211/ZD1211B USB-wireless support
[ ]   ZyDAS ZD1211 debugging
[*] Quantenna wireless cards support
<>   Quantenna qsr1000/qsr2000/qsr10g PCIe support
<M> BL616L wireless driver
    BL616L WIFI BLE configuration (WIFI over USB and BLE over USB combo.) --->
<M> Simulated Radio testing tool for mac80211
<M> Wireless RNDIS USB support
<>   Wifi wrapper for ethernet drivers

```

<Select> < Exit > < Help > < Save > < Load >